

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

NAME: ANGEL DEEPIKA M

COURSE CODE: CSA-0404

REG NO: 192521272

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 100

CASE STUDY:

1. Describe the complete address translation mechanism using base and limit registers in the given system. Explain how this method ensures memory protection and prevents processes from accessing unauthorized memory locations.
2. Explain the concept of paging as implemented in the system. Describe how logical memory is divided into pages and mapped to physical frames. Discuss how paging helps in avoiding external fragmentation and improving memory utilization.
3. Describe the working of the Translation Lookaside Buffer (TLB) in the given system. Explain how TLB reduces memory access time and improves overall system performance.
4. Differentiate between TLB hit and TLB miss in detail. Explain how each case affects memory access time and system efficiency, and illustrate with an example.
5. Describe the concept of segmentation in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the benefits of segmentation over other memory management techniques.
6. Explain the structure and functioning of a segment table. Describe how logical addresses are translated into physical addresses using segment number and offset.
7. Compare and contrast paging and segmentation in detail. Discuss their differences in terms of memory allocation, fragmentation, flexibility, and performance.
8. Explain the concept of virtual memory and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.
9. Describe the page fault handling process in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

10. Explain the need for page replacement algorithms in virtual memory systems. Describe the working of the FIFO page replacement algorithm with a suitable example.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Address Translation Using Base and Limit Registers

Base and limit registers are used to translate logical addresses into physical addresses while providing memory protection.

- **Base register** contains the starting physical address allocated to a process.
- **Limit register** contains the size or maximum valid logical address range of the process.
- When a process generates a logical address, the OS first checks whether:

Logical Address < Limit Register

- If valid, the physical address is calculated as:

Physical Address = Base Register + Logical Address

- If the logical address is greater than or equal to the limit, the hardware generates a protection fault and denies access.

Example:

Base = 5000, Limit = 1000, Logical Address = 400

Physical Address = 5000 + 400 = **5400**

Since $400 < 1000$, the access is valid.

Thus, base and limit registers prevent a process from accessing memory belonging to other processes or the operating system.

2. Paging

Paging is a memory management technique in which logical memory is divided into fixed-size pages and physical memory is divided into equal-sized frames.

When a process runs, its pages can be loaded into any available physical frames. A page table maintains the mapping between page numbers and frame numbers.

A logical address consists of:

Logical Address = Page Number + Offset

The page number is used to find the corresponding frame in the page table. The physical address is then obtained using:

$$\text{Physical Address} = \text{Frame Number} \times \text{Page Size} + \text{Offset}$$

Paging eliminates external fragmentation because any free frame can hold any page. It also improves memory utilization because pages do not need to be stored continuously.

However, paging may cause small amounts of internal fragmentation, especially in the last page.

3. Translation Lookaside Buffer (TLB)

A Translation Lookaside Buffer (TLB) is a small, fast cache that stores recently used page number → frame number mappings.

When the CPU generates a logical address, the page number is first searched in the TLB.

TLB Hit

If the page number is found, the corresponding frame number is obtained immediately. This avoids accessing the page table in main memory.

TLB Miss

If the page number is not found, the system accesses the page table in main memory to obtain the frame number. The mapping may then be stored in the TLB for future use.

Therefore, TLB significantly reduces address translation time because frequently used mappings can be obtained much faster than from the page table.

Overall effect: Higher TLB hit rate → fewer memory accesses → lower average memory access time → better system performance.

4. TLB Hit vs TLB Miss

TLB Hit

Required page number is found in TLB.

Frame number is obtained quickly.

Faster memory access.

Requires fewer memory accesses.

Improves CPU performance.

TLB Miss

Required page number is not found in TLB.

Page table must be accessed.

Slower memory access.

Requires additional memory access.

Reduces performance temporarily.

Example

Suppose a process generates page number 5.

- If page 5 is present in the TLB and maps to frame 12, it is a TLB hit, so the system immediately uses frame 12.
- If page 5 is not present, it is a TLB miss. The system searches the page table, finds frame 12, and may update the TLB.

Thus, a high TLB hit ratio improves system efficiency and reduces the average memory access time.

5. Segmentation

Segmentation is a memory management technique in which a program is divided into logical units called segments according to their purpose.

Common segments include:

- **Code segment** – contains program instructions.
- **Data segment** – contains global and static data.
- **Stack segment** – contains function calls and local variables.
- **Heap segment** – contains dynamically allocated memory.

Each segment has a different size and is identified by a segment number.

Segmentation provides logical organization and allows different protection permissions for different segments. For example, the code segment can be made read-only while the data segment can be read/write.

Unlike paging, segmentation uses variable-sized memory blocks, making it closely related to the programmer's logical view of a program.

However, variable-sized allocation can cause external fragmentation. Still, segmentation provides better logical organization and protection.

6. Segment Table

A **segment table** stores information about each segment of a process. Each entry generally contains:

- **Base** – starting physical address of the segment.
- **Limit** – length or size of the segment.
- Protection information such as read/write/execute permissions.

A logical address contains:

Logical Address = Segment Number + Offset

Translation Process

1. CPU generates a segment number and offset.
2. Segment number is used to index the segment table.
3. The OS/hardware checks whether:

Offset < Limit

4. If valid:

Physical Address = Base + Offset

5. If the offset exceeds the limit, a **segmentation fault/protection fault** occurs.

Example:

Segment Base = 4000

Limit = 1000

Offset = 300

Physical Address = 4000 + 300 = **4300**

Since $300 < 1000$, the address is valid.

7. Paging vs Segmentation

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
Based on	Physical memory management	Logical program structure
Address	Page number + offset	Segment number + offset
Fragmentation	Mainly internal fragmentation	Mainly external fragmentation
Allocation	Any free frame can be used	Requires suitable contiguous space
Programmer view	Less visible	Closely matches program structure
Protection	Page-level protection	Segment-level protection
Flexibility	Good for efficient memory allocation	Good for logical organization

Analysis: Paging is generally better for efficient physical memory utilization, while segmentation provides better logical organization and protection. Modern systems often use paging extensively because it avoids external fragmentation.

8. Virtual Memory

Virtual memory is a memory management technique that allows a process to execute even when its entire program is not loaded into physical RAM.

Only the required pages are loaded into memory. The remaining pages stay in **secondary storage**, such as a disk.

When a required page is not present in RAM, a **page fault** occurs. The operating system loads the required page from secondary storage into an available frame.

Importance

- Allows programs larger than physical RAM to execute.
- Increases the number of processes that can be kept in memory.
- Provides process isolation and protection.
- Improves memory utilization.

For example, if a program requires 8 GB of virtual memory but only part of it is actively used, the system does not need to keep the entire 8 GB in RAM simultaneously.

Thus, virtual memory gives processes the appearance of having a large continuous memory space.

9. Page Fault Handling

A **page fault** occurs when a process tries to access a page that is not currently present in physical memory.

Steps involved:

1. CPU generates a logical address.
2. The page table is checked to determine whether the page is in memory.
3. If the page is absent, a page fault trap is generated.
4. The operating system checks whether the memory reference is valid.
5. It locates the required page in secondary storage.

6. A free frame is selected. If no free frame exists, a page replacement algorithm selects a page to remove.
7. The required page is loaded into the selected frame.
8. The page table is updated with the new frame number.
9. The interrupted instruction is restarted.

Example: If page 4 is required but is not in RAM, the OS loads page 4 from disk into a frame and then continues execution.

Page faults are necessary for virtual memory but are relatively slow because disk access is much slower than RAM access.

10. FIFO Page Replacement Algorithm

Page replacement algorithms are required when a page fault occurs and **no free** frame is available. They determine which page should be removed from memory.

FIFO (First-In, First-Out) replaces the page that has been in memory for the longest time.

Example

Consider **3 frames** and the reference string:

1, 2, 3, 4

Reference Frames		Result
1	1	Page Fault
2	1, 2	Page Fault
3	1, 2, 3	Page Fault
4	4, 2, 3	Page Fault – 1 removed

Here, page **1** was loaded first, so FIFO removes it when page 4 is required.

Advantages

- Simple to understand and implement.
- Requires little overhead.

Disadvantage

FIFO may remove an important frequently used page simply because it is old. It can also suffer from **Be lady's anomaly**, where increasing the number of frames can sometimes increase the number of page faults.

Conclusion: FIFO is simple but may not always provide the best memory performance.